Modern Data Architecture, Polyglot Persistence, and Advanced SQL Ingestion

Created by Dr. David Gomillion with assistance from AI

Executive Summary & Core Concepts

As enterprise data ecosystems grow in volume, velocity, and variety, the traditional "one-size-fits-all" relational database model can no longer serve every analytical, operational, and artificial intelligence (AI) requirement. Modern data engineers and enterprise architects must master **polyglot persistence**—the practice of combining specialized storage engines to handle distinct data modalities—while maintaining rigorous standards for data quality, consistency, and ingestion efficiency.

This reading covers the operational and architectural foundations of polyglot persistence alongside advanced SQL mechanisms used for bulk ingestion, staging, and programmatic data manipulation.

Core Learning Objectives

1. **Architectural Trade-offs:** Distinguish between Online Transaction Processing (OLTP), Online Analytical Processing (OLAP), and Vector Databases in storage layout, access patterns, and indexing.
2. **Enterprise Multi-Engine Pipelines:** Understand how different data streams (relational ERP, IoT telemetry, unstructured text, and graph networks) fit into a modern enterprise data architecture like Project Gigavolt.
3. **Bulk Ingestion & Staging:** Apply high-speed data loading (LOAD DATA INFILE) alongside staging tables to process raw enterprise payloads safely.
4. **Programmatic SQL:** Utilize Common Table Expressions (CTEs), including recursive queries for hierarchical structures, and Window Functions for deduplication and time-series analysis.

Part 1: Modern Data Architecture & Polyglot Persistence

1.1 Beyond the Monolithic Relational Database

For decades, the relational database management system (RDBMS) governed by **ACID** (Atomicity, Consistency, Isolation, Durability) guarantees served as the single standard for data storage. While platforms like MariaDB, MySQL, and PostgreSQL excel at handling structured operational transactions, modern enterprise workloads introduce challenges that stretch a standard row-based RDBMS beyond its optimal capabilities:

- **Unstructured Text & AI Retrieval:** Diagnostic logs, PDFs, and technician notes require semantic understanding via machine learning models and high-dimensional vector similarity rather than basic keyword matching (LIKE '%fault%').

- **High-Velocity Sensor Telemetry:** Industrial IoT devices emit continuous streams of JSON logs. Writing millions of sensor reads per minute into row-oriented B-tree indexes creates heavy write amplification and lock contention.
- **Complex Hierarchies & Networks:** Multi-tier supply chains ("Tier 3 Supplier -> Tier 2 Component -> Tier 1 Plant") require traversing graph structures. Executing deeply nested self-joins in standard relational tables degrades query performance rapidly as depth increases.

Modern enterprise architecture addresses these varied requirements through **polyglot persistence**: selecting and integrating storage engines tailored specifically to the mathematical nature of each workload.

Table 1: Architectural comparison of storage engine modalities across OLTP, OLAP, and Vector engines.

Architectural Dimension	**OLTP (Relational Row-Store)**	**OLAP (Columnar Engine)**	**Vector Database (Dense Embedding Engine)**
Primary Workload	Single-record reads and updates (e.g., Customer Checkout)	Large-scale aggregations over billions of rows (e.g., Annual Financial Reports)	Nearest Neighbor similarity search (e.g., Semantic RAG Search)
On-Disk Layout	Row-oriented (Contiguous bytes per record)	Column-oriented (Contiguous blocks per attribute)	High-dimensional dense arrays (e.g., 384 or 768 float32 values)
Indexing Mechanics	B-Tree / B+Tree indexes for key-value point lookups	Min/Max Extent metadata and dictionary encoding	Graph-based Approximate Nearest Neighbor (e.g., HNSW, IVF)
Optimization Focus	Transactional safety and low-latency point reads	Reduced read I/O and high columnar data compression	Fast geometric distance calculations (Cosine, L2)

1.2 Enterprise Context: Project Gigavolt

Throughout your project milestones, you will work with **Project Gigavolt**—an enterprise multi-engine architecture for a manufacturer of commercial energy storage systems. Managing this infrastructure requires ingesting and harmonizing four distinct operational data streams across dedicated storage engines.

Table 2: Enterprise data modalities, ingestion targets, and processing engines for Project Gigavolt.

Stream / Modality	Sample Data Payload	Primary Processing Strategy	Target Storage Engine
Operational ERP Data	Transactional CSVs (gigavolt_erp.csv)	Staging tables, regex cleansing, SQL type parsing	**MariaDB 3NF Core** (Relational)
Engineering Documentation	SOP PDFs & Manuals (/manuals/*.pdf)	Text chunking, embedding generation	**HNSW Vector Store** (pgvector / Chroma)
IoT Sensor Streams	BMS Telemetry Logs (bms_telemetry.json)	Streaming JSON parsing, document indexing	**Document / NoSQL Telemetry Store**
Supply Chain Network	Supplier Dependencies (suppliers_graph.csv)	Recursive graph traversal, risk mapping	**Graph Engine** (Neo4j / SQL Graph CTEs)

Early project milestones focus on building the foundational relational pipeline: loading raw, unformatted ERP payloads into staging environments, cleansing dirty data fields, and populating normalized production tables.

Part 2: High-Throughput Ingestion & Staging Patterns

2.1 Bulk Ingestion (LOAD DATA INFILE)

Loading large enterprise datasets using individual row-by-row INSERT statements introduces massive performance bottlenecks due to repeated query parsing, transaction logging overhead, and continuous B-tree re-indexing.

To achieve maximum write throughput, database engines provide direct bulk-loading utilities. MariaDB's LOAD DATA INFILE command streams text files directly into the database engine, bypassing application-level parsing.

Table 3: Comparison of row-by-row insertion versus bulk data loading.

Ingestion Method	Execution Mechanism	Performance Impact	Best Use Case
Row-by-Row INSERT	Individual SQL statements sent sequentially over a network connection	High network latency and heavy transaction log overhead	Single-record application updates

Bulk LOAD DATA INFILE	Direct raw block streaming from local or client storage into the engine	Bypasses parser overhead and optimizes disk batch writes	Initial data loads and large file imports

Key Syntax Parameters

- **LOCAL:** Indicates that the input file resides on the client machine rather than the server's local file system.
- **FIELDS TERMINATED BY ',':** Specifies the column delimiter (e.g., comma, tab).
- **OPTIONALLY ENCLOSED BY '"':** Prevents commas within string fields (such as addresses or notes) from breaking column boundaries.
- **IGNORE 1 LINES:** Skips header rows containing field names.

Security Note: In standard MariaDB installations, client-side bulk file loading (local_infile) is disabled by default to prevent unauthorized file access. Server configurations must explicitly enable this capability before running client-side bulk operations.

2.2 The Staging Table Pattern

Loading unvalidated external files directly into production relational tables with strict foreign keys and check constraints risks job failures and partial data corruption. The **Staging Table Pattern** separates raw file ingestion from data cleansing and validation.

Table 4: Multi-stage pipeline architecture for processing raw enterprise data into production tables.

Pipeline Layer	Engine / Table Structure	Operational Role	Data Quality State
1. Source Payload	CSV File / External Stream	Direct extract from legacy systems	Unvalidated, potential format errors
2. Staging Layer	Broad VARCHAR / TEXT Table	Receives raw bulk import without enforcing constraints	Identical to source file format
3. Cleansing Layer	SQL Transformation Logic	Applies STR_TO_DATE, regex cleanup, string trimming	Cleaned, validated, and formatted
4. Production Core	Normalized 3NF Relational Tables	Stores cleansed records with full constraints enforced	Production-ready, consistent state

Staging Implementation Strategy

1. **Create an Unconstrained Staging Table:** Mirror the layout of the incoming text file using flexible data types (VARCHAR, TEXT) and no primary or foreign key constraints.
2. **Execute Bulk Load:** Stream the raw file into the staging table using LOAD DATA LOCAL INFILE.
3. **Cleanse and Transform via SQL:** Use in-database string and date functions to standardise fields before populating core 3NF tables:
    - Standardize timestamps using STR_TO_DATE().
    - Strip HTML tags or invalid characters using REGEXP_REPLACE().
    - Clean financial values by stripping currency symbols (REPLACE()) and casting to decimal types (CAST()).

Part 3: Programmatic SQL: CTEs & Window Functions

3.1 Common Table Expressions (CTEs)

A **Common Table Expression (CTE)** defines a named temporary result set within the scope of a single SELECT, INSERT, UPDATE, or DELETE statement. Defined using the WITH keyword, CTEs improve readability and modularity compared to nested subqueries.

Non-Recursive CTEs

Non-recursive CTEs break complex, multi-stage queries into logical steps:

```
WITH HighSpendingCustomers AS (
  SELECT
    customer_id,
    SUM(repair_cost) AS total_spend
  FROM core_service_logs
  GROUP BY customer_id
  HAVING SUM(repair_cost) > 5000.00
),
PendingClaims AS (
  SELECT
    claim_id,
    customer_id,
    claim_amount
  FROM core_warranty_claims
  WHERE claim_status = 'Pending'
)
```

```
SELECT
  hsc.customer_id,
  hsc.total_spend,
  pc.claim_id,
  pc.claim_amount
FROM HighSpendingCustomers hsc
JOIN PendingClaims pc ON hsc.customer_id = pc.customer_id;
```

Recursive CTEs for Hierarchical Data

Hierarchical datasets—such as organizational charts or product bill-of-materials (BOM)—cannot be queried efficiently with standard static joins. A **Recursive CTE** self-references to traverse parent-child relationships iteratively.

A recursive CTE requires three components:

1. **Anchor Member:** Selects the top-level root node(s) of the hierarchy.
2. **Recursive Member:** Joins the child table back to the CTE result set.
3. **Termination Condition:** Automatically stops execution when no new child rows are returned.

```
WITH RECURSIVE ComponentHierarchy AS (
  -- Anchor: Find top-level assembly
  SELECT
    component_id,
    parent_component_id,
    component_name,
    1 AS level_depth
  FROM core_components
  WHERE parent_component_id IS NULL

  UNION ALL

  -- Recursive Step: Join sub-components to parent components
  SELECT
    c.component_id,
```

```
    c.parent_component_id,
    c.component_name,
    ch.level_depth + 1
  FROM core_components c
  JOIN ComponentHierarchy ch ON c.parent_component_id = ch.component_id
)
SELECT * FROM ComponentHierarchy ORDER BY level_depth;
```

3.2 SQL Window Functions

Standard aggregate functions (SUM(), AVG()) used with GROUP BY collapse multiple records into a single summary row. **Window Functions** perform calculations across a defined set of related rows while **retaining the full detail of every individual row**.

Table 5: Functional distinctions between GROUP BY aggregations and Window Functions.

Operation	Query Structure	Row Detail Retention	Typical Application
GROUP BY Aggregation	SELECT dept, SUM(sales) GROUP BY dept	Rows collapsed into one row per group	Executive summaries and high-level reports
Window Function	SELECT sales, SUM(sales) OVER (PARTITION BY dept)	Every row remains distinct in output	Deduplication, moving averages, relative ranking

Window Function Syntax

```
FUNCTION() OVER (
  [PARTITION BY partition_column]
  [ORDER BY sort_column]
  [ROWS BETWEEN frame_start AND frame_end]
)
```

1. Ranking Functions & Deduplication

Ranking functions assign integer sequences to rows based on partition boundaries:

- ROW_NUMBER(): Assigns a unique, sequential integer to every row (e.g., 1, 2, 3, 4). Useful for deterministic deduplication.
- RANK(): Assigns identical ranks to tied rows, skipping subsequent numbers (e.g., 1, 2, 2, 4).
- DENSE_RANK(): Assigns identical ranks to tied rows without gaps in sequence (e.g., 1, 2, 2, 3).

Table 6: Comparison of ROW_NUMBER, RANK, and DENSE_RANK on duplicate values.

Customer ID	Service Date	Repair Cost	ROW_NUMBER()	RANK()	DENSE_RANK()
101	2026-08-01	$500.00	1	1	1
101	2026-08-15	$300.00	2	2	2
101	2026-08-15	$300.00	3	2	2
101	2026-08-20	$150.00	4	4	3

To deduplicate staging records while retaining the most recent entry, wrap a ROW_NUMBER() function inside a CTE and filter for row_num = 1:

```
WITH RankedStaging AS (
  SELECT
    raw_dispatch_id,
    raw_equipment_serial,
    raw_dispatch_timestamp,
    ROW_NUMBER() OVER (
      PARTITION BY raw_equipment_serial, raw_dispatch_timestamp
      ORDER BY raw_dispatch_id DESC
    ) AS dedup_rank
  FROM stg_gigavolt_erp
)
SELECT * FROM RankedStaging WHERE dedup_rank = 1;
```

2. Value Navigation (LAG and LEAD)

LAG() and LEAD() access values from preceding or succeeding rows within a partition without needing self-joins.

Table 7: Time-series telemetry tracking using LAG to compute voltage drops.

Telemetry Timestamp	Current Voltage (V)	LAG(Voltage, 1)	Voltage Delta	Anomaly Flag
11:40:00 AM	52.4	NULL	NULL	Baseline
11:40:05 AM	52.1	52.4	-0.3	Normal
11:40:10 AM	48.2	52.1	-3.9	**Voltage Drop Spike**
11:40:15 AM	48.0	48.2	-0.2	Normal

```
SELECT
  device_serial,
  telemetry_timestamp,
  cell_voltage,
  LAG(cell_voltage, 1) OVER (
    PARTITION BY device_serial
    ORDER BY telemetry_timestamp ASC
  ) AS previous_voltage
FROM core_bms_telemetry;
```

Part 4: Practical SQL Reference

Below is a complete SQL script demonstrating staging table setup, bulk loading simulation, data cleansing, deduplication, and window function analytics.

```
-- ============================================================================
-- ISTM 622: Integrated Laboratory Script
-- Focus: Staging Ingestion, Cleansing, CTEs, and Window Analytics
-- Engine: MariaDB / PostgreSQL
-- ============================================================================

CREATE DATABASE IF NOT EXISTS gigavolt_sandbox;
```

```
USE gigavolt_sandbox;

-- 1. Create Staging Table
DROP TABLE IF EXISTS stg_gigavolt_erp;
CREATE TABLE stg_gigavolt_erp (
    raw_dispatch_id VARCHAR(100),
    raw_customer_email VARCHAR(255),
    raw_customer_name VARCHAR(255),
    raw_equipment_serial VARCHAR(100),
    raw_dispatch_timestamp VARCHAR(100),
    raw_repair_cost VARCHAR(100),
    raw_technician_notes TEXT
);

-- 2. Populate Staging Table (Simulating Bulk Load)
INSERT INTO stg_gigavolt_erp VALUES
('DSP-101', 'A.SMITH@CLIENT.COM', 'Alice Smith', 'GIGA-BATT-001', '08/24/2026 08:30 AM', '$1,250.00', 'Replaced <p>cell block B</p>.'),
('DSP-102', 'B.JOHNSON@ENERGY.ORG', 'Bob Johnson', 'GIGA-BATT-002', '08/24/2026 09:15 AM', '$3,400.50', 'Thermal spike observed.'),
('DSP-101', 'A.SMITH@CLIENT.COM', 'Alice Smith', 'GIGA-BATT-001', '08/24/2026 08:30 AM', '$1,250.00', 'DUPLICATE RECORD.'),
('DSP-103', 'A.SMITH@CLIENT.COM', 'Alice Smith', 'GIGA-BATT-001', '08/25/2026 02:00 PM', '$450.00', 'Routine check.');

-- 3. Production Core Schema Setup
DROP TABLE IF EXISTS core_service_dispatches;
DROP TABLE IF EXISTS core_customers;

CREATE TABLE core_customers (
    customer_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    customer_email VARCHAR(150) NOT NULL UNIQUE,
    customer_name VARCHAR(150) NOT NULL
);

CREATE TABLE core_service_dispatches (
    dispatch_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    equipment_serial VARCHAR(50) NOT NULL,
    dispatch_timestamp DATETIME NOT NULL,
    repair_cost DECIMAL(10,2) NOT NULL,
    clean_technician_notes TEXT,
    CONSTRAINT fk_dispatch_customer FOREIGN KEY (customer_id) REFERENCES
core_customers(customer_id)
);

-- 4. Populate Core Customers
INSERT INTO core_customers (customer_email, customer_name)
SELECT DISTINCT
    LOWER(TRIM(raw_customer_email)),
    TRIM(raw_customer_name)
FROM stg_gigavolt_erp;

-- 5. Cleanse, Deduplicate, and Populate Core Dispatches
WITH CleansedStaging AS (
    SELECT
        CAST(REPLACE(raw_dispatch_id, 'DSP-', '') AS SIGNED) AS dispatch_id,
        LOWER(TRIM(raw_customer_email)) AS clean_email,
        TRIM(raw_equipment_serial) AS clean_serial,
        STR_TO_DATE(raw_dispatch_timestamp, '%m/%d/%Y %h:%i %p') AS clean_timestamp,
        CAST(REPLACE(REPLACE(raw_repair_cost, '$', ''), ',', '') AS DECIMAL(10,2)) AS clean_cost,
```

```
    REGEXP_REPLACE(raw_technician_notes, '<[^>]+>', '') AS sanitized_notes,
    ROW_NUMBER() OVER (
      PARTITION BY raw_dispatch_id
      ORDER BY raw_dispatch_timestamp DESC
    ) AS dedup_rank
  FROM stg_gigavolt_erp
)
INSERT INTO core_service_dispatches (dispatch_id, customer_id, equipment_serial,
dispatch_timestamp, repair_cost, clean_technician_notes)
SELECT
  cs.dispatch_id,
  c.customer_id,
  cs.clean_serial,
  cs.clean_timestamp,
  cs.clean_cost,
  cs.sanitized_notes
FROM CleansedStaging cs
JOIN core_customers c ON cs.clean_email = c.customer_email
WHERE cs.dedup_rank = 1;

-- 6. Analytical Query using Window Functions
SELECT
  d.dispatch_id,
  c.customer_name,
  d.equipment_serial,
  d.repair_cost,
  DENSE_RANK() OVER (
    PARTITION BY d.customer_id
    ORDER BY d.repair_cost DESC
  ) AS cost_rank,
```

```
    d.repair_cost - LAG(d.repair_cost, 1, 0.00) OVER (
        PARTITION BY d.customer_id
        ORDER BY d.dispatch_timestamp ASC
    ) AS cost_diff_from_prior,
    SUM(d.repair_cost) OVER (
        PARTITION BY d.customer_id
        ORDER BY d.dispatch_timestamp ASC
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total_spend
FROM core_service_dispatches d
JOIN core_customers c ON d.customer_id = c.customer_id;
```

Review & Preparation Questions

1. **Architecture:** Why is a columnar engine like MariaDB ColumnStore more efficient for running aggregate queries over billions of telemetry rows than a standard row-oriented InnoDB engine?
2. **Ingestion & Integrity:** Why should raw bulk data loads land in an unconstrained staging table before being inserted into production 3NF relational schemas?
3. **Analytics:** How does ROW_NUMBER() OVER (PARTITION BY ...) enable precise deduplication compared to using SELECT DISTINCT?
4. **Recursive SQL:** What component in a Recursive CTE stops execution and prevents an infinite loop when querying hierarchical trees?

References & Further Reading

- **Codd, E. F. (1970).** "A Relational Model of Data for Large Shared Data Banks." *Communications of the ACM*, 13(6), 377–387.
- **MariaDB Corporation.** "LOAD DATA INFILE Syntax and Usage Guide." *MariaDB Knowledge Base*.
- **MariaDB Corporation.** "Window Functions Overview." *MariaDB Knowledge Base*.
- **Kleppmann, M. (2017).** *Designing Data-Intensive Applications*. O'Reilly Media.